

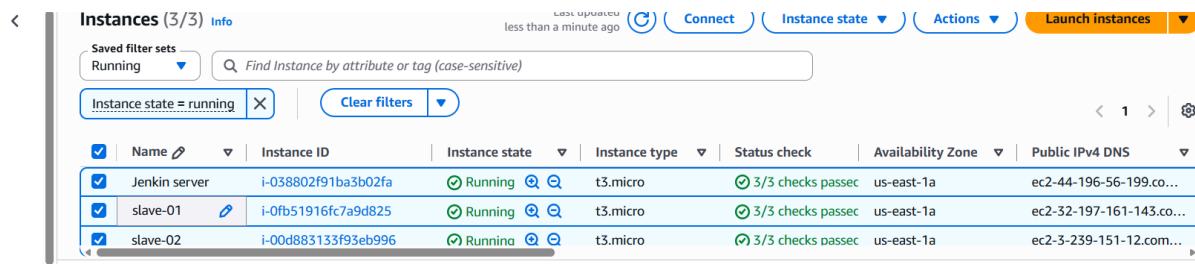
JENKINS-02

1. Configure 2 slave machines in Jenkins master.

EC2 & Initial Setup

1. Launched three EC2 instances:

- jenkins-master
- slave-01
- slave-02



The screenshot shows the AWS Management Console 'Instances' page. It displays three EC2 instances in a 'Running' state. The instances are named 'Jenkin server', 'slave-01', and 'slave-02'. All three are using the 't3.micro' instance type and are located in the 'us-east-1a' availability zone. The 'Jenkin server' has a public IP of 'ec2-44-196-56-199.co...' and a public DNS of 'ec2-44-196-56-199.co...'. 'slave-01' has a public IP of 'ec2-32-197-161-143.co...' and a public DNS of 'ec2-32-197-161-143.co...'. 'slave-02' has a public IP of 'ec2-3-239-151-12.com...' and a public DNS of 'ec2-3-239-151-12.com...'. All three instances have a status check of '3/3 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
Jenkin server	i-038802f91ba3b02fa	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-44-196-56-199.co...
slave-01	i-0fb51916fc7a9d825	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-32-197-161-143.co...
slave-02	i-00d883133f93eb996	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-3-239-151-12.com...

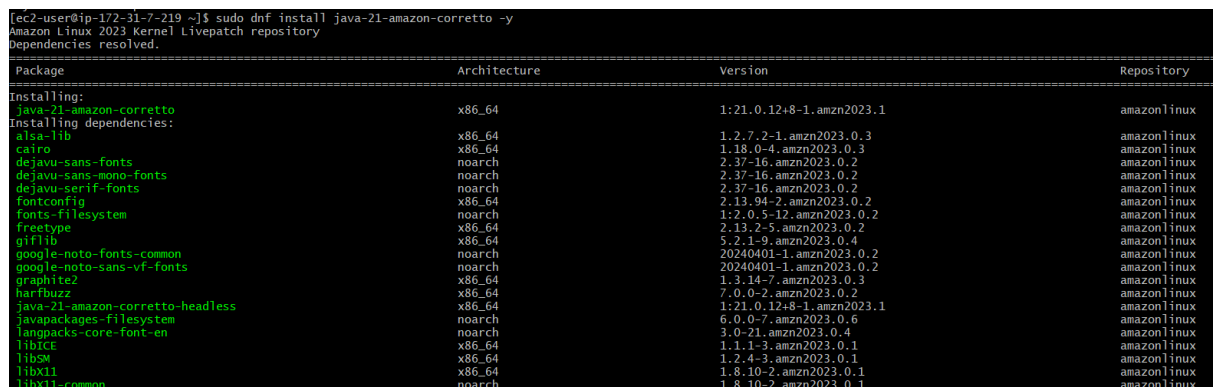
Slave-01 OS Configuration

Logged in to slave-01 as root.

Installed Java:

```
sudo yum install java-21-amazon-corretto -y
```

Verified Java installation completed.



The screenshot shows a terminal window with the command 'sudo dnf install java-21-amazon-corretto -y' being executed. The output shows the installation of the 'java-21-amazon-corretto' package and its dependencies. The packages are listed in a table with columns for Package, Architecture, Version, and Repository.

Package	Architecture	Version	Repository
Installing: java-21-amazon-corretto	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux
Installing dependencies:			
alsa-lib	x86_64	1.2.7-2-1.amzn2023.0.3	amazonlinux
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux
freetype	x86_64	2.13.2-5.amzn2023.0.2	amazonlinux
glib	x86_64	5.2.1-9.amzn2023.0.4	amazonlinux
google-noto-fonts-common	noarch	20240401-1.amzn2023.0.2	amazonlinux
google-noto-sans-vf-fonts	noarch	20240401-1.amzn2023.0.2	amazonlinux
graphite2	x86_64	1.3.14-7.amzn2023.0.3	amazonlinux
harfbuzz	x86_64	7.0.0-2.amzn2023.0.2	amazonlinux
java-21-amazon-corretto-headless	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux
javapackages-filesystem	noarch	6.0.0-7.amzn2023.0.6	amazonlinux
langpacks-core-font-en	noarch	3.0-21.amzn2023.0.4	amazonlinux
libICE	x86_64	1.1.1-3.amzn2023.0.1	amazonlinux
libSM	x86_64	1.2.4-3.amzn2023.0.1	amazonlinux
libX11	x86_64	1.8.10-2.amzn2023.0.1	amazonlinux
libX11-common	noarch	1.8.10-2.amzn2023.0.1	amazonlinux

SSH Configuration on Slave-01

Generated SSH key pair:

```
ssh-keygen
```

Navigated to SSH directory:

```
cd ~/.ssh
```

Added public key to authorized keys:

cat id_rsa.pub >> authorized_keys

Fixed permissions:

chmod 700 authorized_keys

```
[ec2-user@ip-172-31-7-219 ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_rsa):
Enter passphrase for "/home/ec2-user/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/id_rsa
Your public key has been saved in /home/ec2-user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:zRbe77hfz2Po0+sk+JTX3ZR8sPWk/h3OR4MmGM078ag ec2-user@ip-172-31-7-219.ec2.internal
The key's randomart image is:
+---[RSA 3072]-----+
|
| . . . o
| B o . *o
| S X . +o+
| = . . . o=
| . =+oo++=
| oo,+o*=
| E. . =BBoB|
+---[SHA256]-----+
[ec2-user@ip-172-31-7-219 ~]$ cd ~/.ssh
[ec2-user@ip-172-31-7-219 .ssh]$ ls
authorized_keys id_rsa id_rsa.pub
[ec2-user@ip-172-31-7-219 .ssh]$ cat id_rsa.pub >> authorized_keys
[ec2-user@ip-172-31-7-219 .ssh]$ chmod 700 authorized_keys
[ec2-user@ip-172-31-7-219 .ssh]$
```

Jenkins Master SSH Trust

Logged in to jenkins-master.

Added slave host key:

ssh-keyscan -H 13.61.10.105 >> /var/lib/jenkins/.ssh/known_hosts

Set correct permissions:

chown jenkins:jenkins /var/lib/jenkins/.ssh/known_hosts

chmod 700 /var/lib/jenkins/.ssh

chmod 600 /var/lib/jenkins/.ssh/known_hosts

```
[root@ip-172-31-4-173 ec2-user]# mkdir -p /var/lib/jenkins/.ssh
[root@ip-172-31-4-173 ec2-user]# cd /var/lib/jenkins/.ssh
bash: cd: /var/lib/jenkins/.ssh: No such file or directory
[root@ip-172-31-4-173 ec2-user]# cd /var/lib/jenkins/.ssh
[root@ip-172-31-4-173 .ssh]# ssh-keyscan -H 32.197.161.143 >> /var/lib/jenkins/.ssh/known_hosts
[root@ip-172-31-4-173 .ssh]# chown jenkins:jenkins known_hosts
[root@ip-172-31-4-173 .ssh]# chmod 700 known_hosts
[root@ip-172-31-4-173 .ssh]#
```

Jenkins Node Creation (slave-01)

Opened Jenkins dashboard.

Navigated to **Manage Jenkins** → **Nodes**.

Clicked **New Node**.

Entered node name: slave-01.

Selected **Permanent Agent** and created the node.

The screenshot shows the Jenkins 'Manage Jenkins' interface. The top navigation bar includes 'Jenkins / Manage Jenkins' and a search bar. The main content area is titled 'Manage Jenkins' and contains a 'System Configuration' section with links to 'System', 'Tools', 'Plugins', 'Nodes', 'Clouds', and 'Appearance'. The 'Nodes' link is highlighted, leading to the 'Nodes' page. This page shows a table of nodes with columns: S, Name, Architecture, Clock Difference, Free Disk Space, Free Swap Space, Free Temp Space, and Response Time. A 'Built-In Node' is listed with a 'Data obtained' row showing 52 min for all metrics. Below the table are tabs for 'Icon', 'S', 'M', and 'L'. The 'New node' page is also shown, with a 'Node name' field containing 'slave-01' and a 'Type' dropdown set to 'Permanent Agent'. A 'Create' button is at the bottom.

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	17.45 GiB	0 B	451.87 MiB	0ms
	Data obtained	52 min	52 min	52 min	52 min	52 min	52 min

Icon: S M L Legend

New node

Node name

slave-01

Type

☒ Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

Node Configuration (slave-01)

Set **Number of executors** to 2.

Initially observed agent offline due to root directory permission issue.

Logged in to slave-01 and created Jenkins workspace:

```
mkdir -p /home/ec2-user/jenkins
```

```
chown -R ec2-user:ec2-user /home/ec2-user/jenkins
```

Updated **Remote root directory** in Jenkins to:

```
/home/ec2-user/jenkins
```

Added **Label**:

```
java
```

Set **Usage** to:

Only build jobs with label expressions matching this node

 Jenkins

Manage Jenkins

Nodes

Search

Settings

User

Name ?

slave-01

Description ?

Plain text

Preview

Number of executors ?

2

Remote root directory ?

/root

Labels ?

java

Usage ?

Only build jobs with label expressions matching this node

Launch method ?

Launch agents via SSH

Host ?

32.197.161.143

Credentials ?

ec2-user

+ Add

Host Key Verification Strategy ?

Known hosts file Verification Strategy

Advanced

Availability ?

Keep this agent online as much as possible

Save

SSH Launch Configuration

Selected **Launch agents via SSH**.

Entered **Host**: 13.61.10.105.

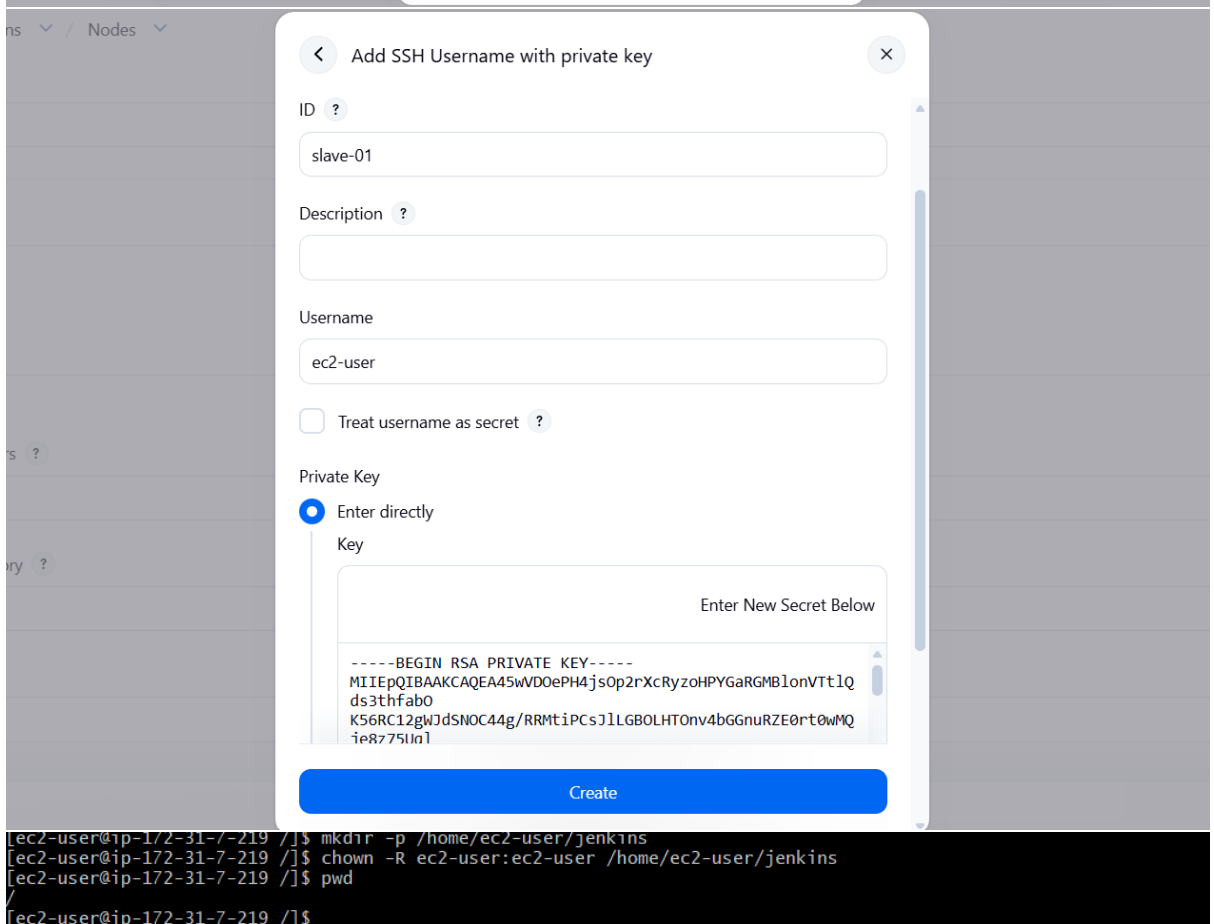
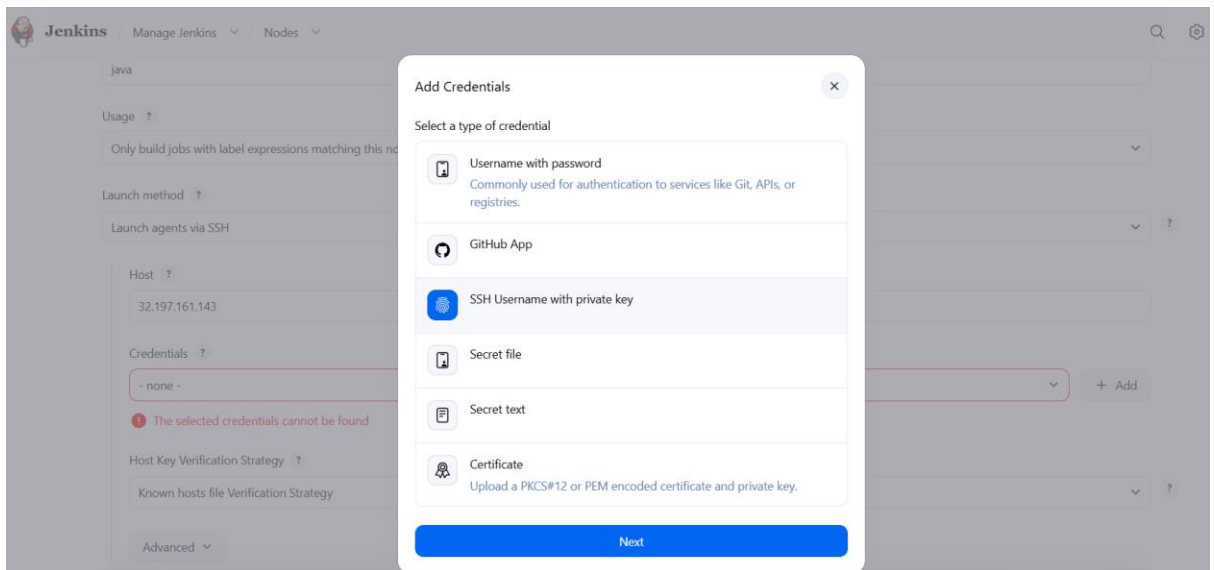
Selected SSH **credentials (ec2-user)**.

Chose **Known hosts file Verification Strategy**.

Saved configuration.

Verified SSH authentication successful in node logs.

Confirmed slave-01 status changed to **online**.



Jenkins / Nodes / slave-01 / Log

Status
Delete Agent
Configure
Build History
Load Statistics
Log

Build Executor Status 0/2
slave-01 (offline)

```
Warning: no key algorithms provided; JENKINS-42959 disabled
SSHLauncher{host='32.197.161.143', port=22, credentialsId='slave-01', jvmOptions='', javaPath='', prefixStartsSlaveCmd='',
suffixStartsSlaveCmd='', launchTimeoutSeconds=60, maxNumRetries=10, retryWaitTime=15,
sshHostKeyVerificationStrategy=hudson.plugins.sshslaves.verifiers.KnownHostsFileKeyVerificationStrategy, tcpNoDelay=true,
trackCredentials=true}
[08/11/26 14:23:22] [SSH] Opening SSH connection to 32.197.161.143:22.
Searching for 32.197.161.143 in /var/lib/jenkins/.ssh/known_hosts
Searching for 32.197.161.143:22 in /var/lib/jenkins/.ssh/known_hosts
[08/11/26 14:23:22] [SSH] SSH host key matches key in Known Hosts file. Connection will be allowed.
[08/11/26 14:23:22] [SSH] Authentication successful.
[08/11/26 14:23:22] [SSH] The remote user's environment is:
BASH=/usr/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:extquote:force_ignore:globasciiranges:globskipdots:hostcomplete:interactive_comments:patsub
_replacement:progcomp:promptvars:sourcepath
BASHRC_SOURCED=Y
BASH_ALIASES=()
BASH_ARGC=([0]="0")
BASH_ARGV=()
BASH_CMDS=()
BASH_EXECUTION_STRING=set
BASH_LINENO=()
```

Jenkins / Nodes / slave-01 / Configure

Status
Delete Agent
Configure
Build History
Load Statistics
Log

Build Executor Status 0/2
slave-01 (offline)

Name ?
slave-01

Description ?

Plain text Preview

Number of executors ?
2

Remote root directory ?
/home/ec2-user/jenkins

Labels ?
java

Save Apply

Jenkins / Nodes

+ New Node Configure Monitors

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	17.45 GiB	0 B	451.87 MiB	0ms
	slave-01	Linux (amd64)	In sync	5.94 GiB	0 B	456.43 MiB	85ms
	Data obtained	4 min 45 sec	4 min 45 sec	4 min 45 sec	4 min 45 sec	4 min 45 sec	4 min 45 sec

Icon: S M L Legend

Job Creation & Execution

Clicked **New Item**.

Entered job name: `first_job`.

Selected **Freestyle project**.

Enabled **Restrict where this project can be run**.

Entered label:

java

Configured **Source Code Management** → **Git**.

Entered repository URL:

`https://github.com/betawins/Techie_horizon_Login_app.git`

Saved the job configuration.

Ran the job.

Verified build executed **remotely on slave-01**.

Confirmed **Git clone successful**.

Build finished with **SUCCESS**.

Final Node Verification

Navigated to **Manage Jenkins** → **Nodes**.

Verified:

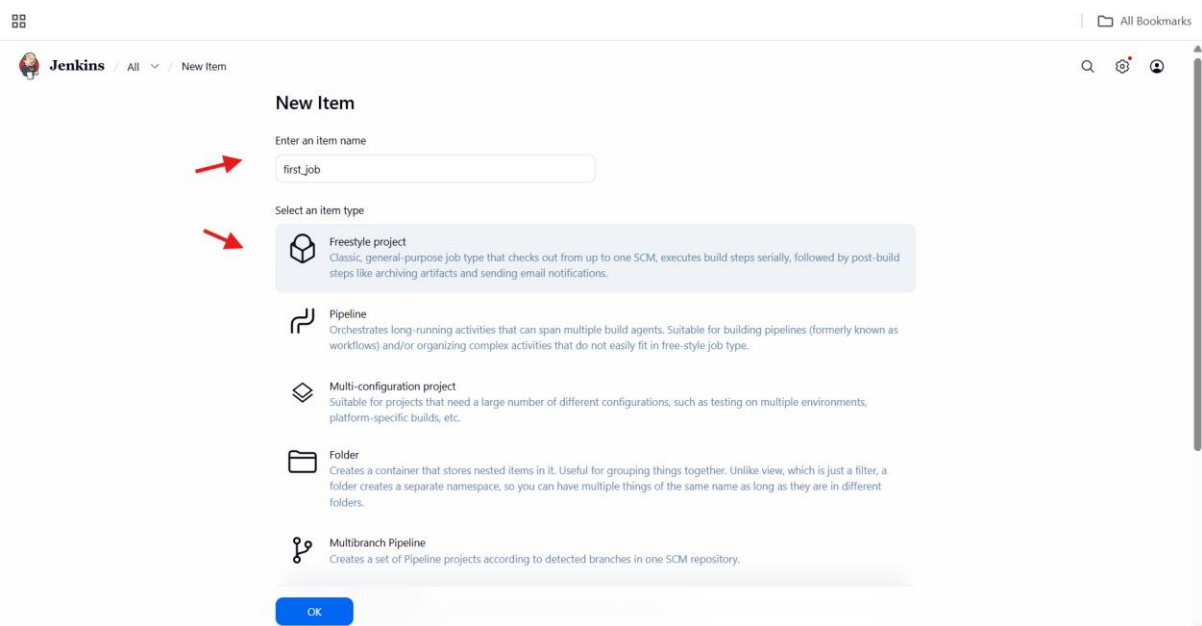
- Built-in node → online
- slave-01 → online with disk & response metrics visible

Slave-02 Configuration

Repeated **the same configuration steps** (Java install, SSH setup, workspace creation, Jenkins node setup) for **slave-02**.

Added slave-02 successfully to Jenkins.

Verified both **slave-01** and **slave-02** are **online and available for builds**.



Jenkins

first-job / Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Restrict where this project can be run

Label Expression

java

Label java matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

None

Git

Repositories

Repository URL

https://github.com/betawins/Techie_horizon_Login_app.git

Credentials

- none -

Add

Advanced

Save

Apply

Jenkins

first-job / #4

Status

Changes

Console Output

Delete build '#4'

Git Build Data

Edit build information

Previous Build

Console

Download

Copy

View as plain text

Started by user BOORA SRIHARI

Running as SYSTEM

Building remotely on slave-01 (java) in workspace /home/ec2-user/jenkins/workspace/first-job

The recommended git tool is: NONE

No credentials specified

Cloning the remote Git repository

Cloning repository https://github.com/betawins/Techie_horizon_Login_app.git

> git init /home/ec2-user/jenkins/workspace/first-job # timeout=10

Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git

> git --version # timeout=10

> git --version # 'git version 2.50.1'

> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10

> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10

Jenkins

Nodes

+ New Node

Configure Monitors

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	17.45 GiB	0 B	451.87 MiB	0ms
	slave-01	Linux (amd64)	In sync	5.94 GiB	0 B	456.40 MiB	41ms
	slave-02	Linux (amd64)	In sync	5.94 GiB	0 B	456.40 MiB	54ms
Data obtained		54 sec	54 sec	54 sec	54 sec	54 sec	54 sec

Icon:

S

M

L

Legend

2.Configure webhooks to Jenkins job.

Jenkins Job Configuration

Opened Jenkins and selected the job **webhook**.

Clicked **Configure**.

Enabled **Restrict where this project can be run**.

Entered **Label Expression** as:

Master

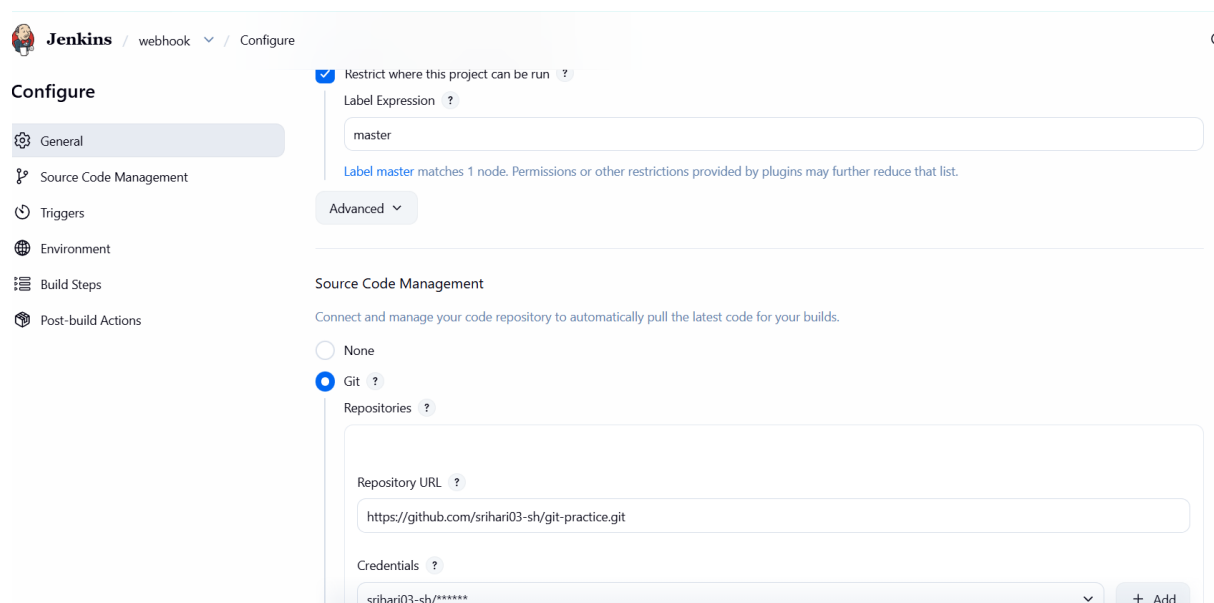
Source Code Management Setup

Selected **Git** under Source Code Management.

Entered **Repository URL**:

<https://github.com/srihari03-sh/git-practice.git>

Left **Credentials** as none (public repository).



The screenshot shows the Jenkins 'Configure' page for a job named 'webhook'. The left sidebar contains a 'Configure' section with a list of tabs: General, Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions. The 'General' tab is selected. The main content area shows the 'Restrict where this project can be run' checkbox checked, with a 'Label Expression' field containing 'master'. Below this is an 'Advanced' dropdown. The 'Source Code Management' section is expanded, showing 'Git' selected as the provider. The 'Repositories' section contains a 'Repository URL' field with the value 'https://github.com/srihari03-sh/git-practice.git' and a 'Credentials' dropdown menu showing 'srihari03-sh/*****'.

Trigger Configuration

Navigated to **Triggers** section.

Enabled:

GitHub hook trigger for GITScm pollin

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Build Step Configuration

Scrolled to **Build Steps**.

Selected **Execute shell**.

Added command:

sleep 15

Clicked **Save**.

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

[See the list of available environment variables](#)

```
sleep 15
```

Advanced ▾

GitHub Webhook Creation

Opened GitHub repository:

<https://github.com/srihari03-sh/git-practice>

Navigated to:

Settings → Webhooks → Add webhook

Entered **Payload URL**:

<http://13.53.212.231:8081/github-webhook/>

Selected **Content type**:

application/json

Kept **Secret** empty.

Selected:

Send me everything

Enabled **Active**.

Clicked **Add webhook**.

Verified webhook added successfully

The image shows two screenshots of the GitHub interface. The top screenshot displays the 'Settings' page for a repository named 'git-practice'. The 'Webhooks' option is selected in the left sidebar. The 'General' tab is active, showing the repository name 'git-practice', a 'Rename' button, and the 'Default branch' set to 'main'. The 'Releases' section shows the 'Enable release immutability' checkbox is unchecked. The bottom screenshot shows the 'Add webhook' form. The 'Payload URL' is 'http://98.86.150.73:8081/github-webhook/'. The 'Content type' is 'application/json'. The 'Secret' field is empty. Under 'SSL verification', 'Enable SSL verification' is selected. Under 'Which events would you like to trigger this webhook?', 'Send me everything.' is selected. The 'Active' checkbox is checked, and the 'Add webhook' button is visible at the bottom.

github.com/srihari03-sh / git-practice

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

General

Repository name

git-practice Rename

Template repository

Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)

Default branch

The default branch is considered the "base" branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

main

Releases

Enable release immutability

Disallow assets and tags from being modified once a release is published.

github.com/srihari03-sh/git-practice/settings/hooks/new

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Webhooks / Add webhook

We'll send a Post request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

http://98.86.150.73:8081/github-webhook/

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification Disable (not recommended)

Which events would you like to trigger this webhook?

Just the push event.

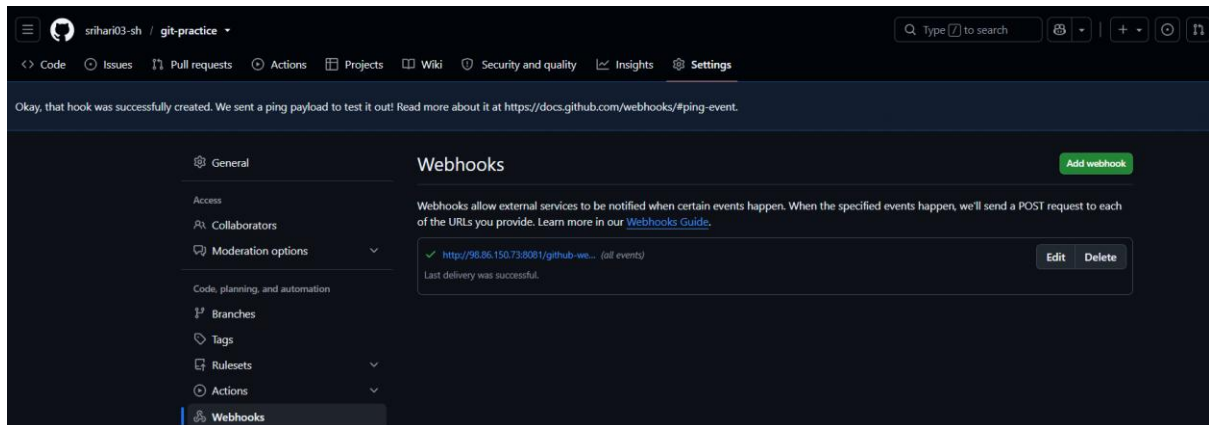
Send me everything.

Let me select individual events.

Active

We will deliver event details when this hook is triggered.

Add webhook

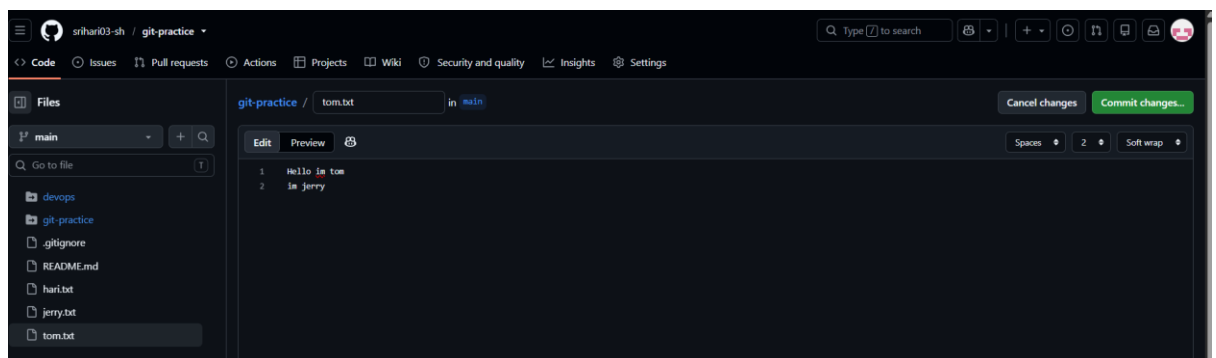


Repository Change

Opened **README.md** in the repository.

Edited the file.

Committed changes to the **master** branch.



Jenkins Build Execution

GitHub webhook triggered Jenkins automatically.

Jenkins job started without manual build.

Build executed on **Built-in (master) node**.

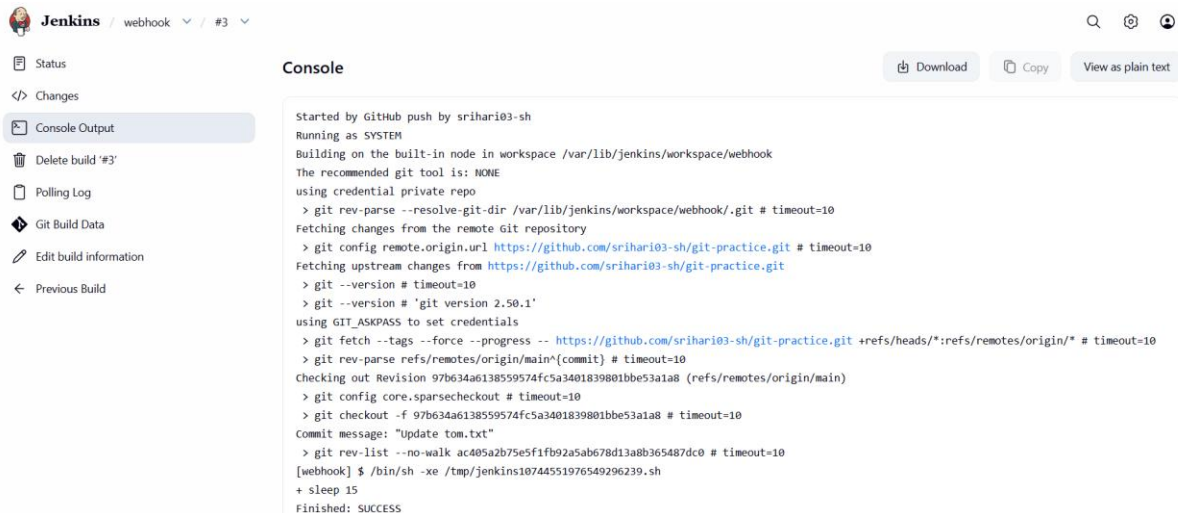
Jenkins cloned the repository successfully.

Shell command executed.

Console output showed:

job triggered successfully

Build finished with **SUCCESS** status.

A screenshot of the Jenkins web interface showing the console output of a build. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build #3, Polling Log, Git Build Data, Edit build information, and Previous Build. The main area is titled 'Console' and shows a series of terminal commands and their outputs. The commands include setting up a git repository, fetching changes, checking out a revision, and committing a file. The build ends with 'Finished: SUCCESS'.

```
Started by GitHub push by srihari03-sh
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/webhook
The recommended git tool is: NONE
using credential private repo
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/webhook/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/srihari03-sh/git-practice.git # timeout=10
Fetching upstream changes from https://github.com/srihari03-sh/git-practice.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/srihari03-sh/git-practice.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 97b634a6138559574fc5a3401839801bbe53a1a8 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 97b634a6138559574fc5a3401839801bbe53a1a8 # timeout=10
Commit message: "Update tom.txt"
> git rev-list --no-walk ac405a2b75e5f1fb92a5ab678d13a8b365487dc0 # timeout=10
[webhook] $ /bin/sh -xe /tmp/jenkins10744551976549296239.sh
+ sleep 15
Finished: SUCCESS
```

3.Configure poll scm and build periodical options in Jenkins job.

Steps Performed – Jenkins Periodic Build (first_job)

Job General Configuration

Opened Jenkins and selected **first_job**.

Clicked **Configure**.

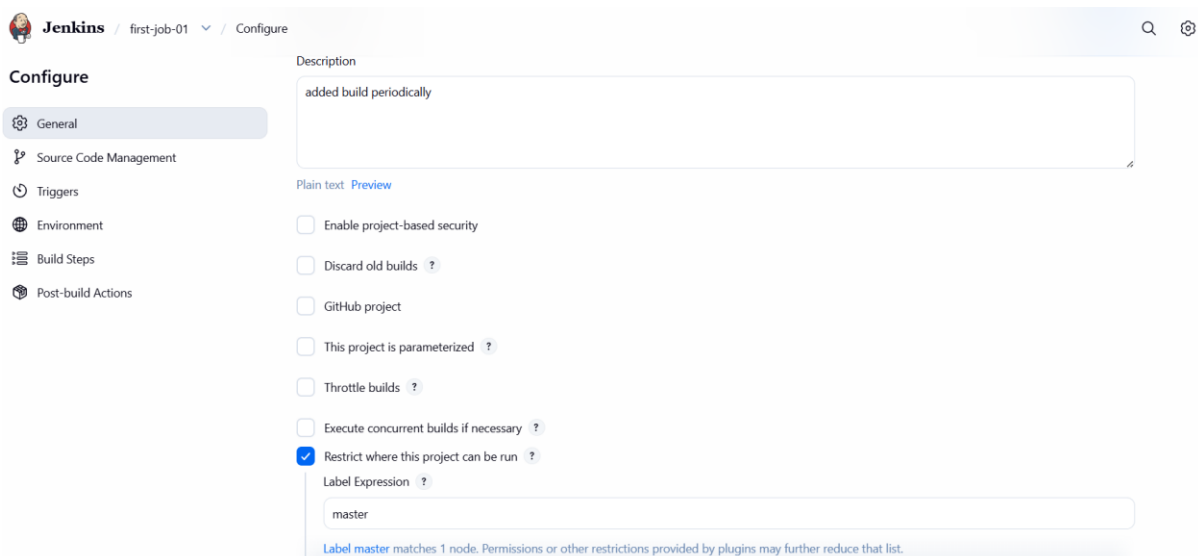
Added **Description**:

added build periodically

Enabled **Restrict where this project can be run**.

Set **Label Expression** to:

master

A screenshot of the Jenkins 'Configure' page for a job named 'first-job-01'. The left sidebar shows the configuration sections: General (selected), Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions. The 'Description' field contains the text 'added build periodically'. Below this, there are several checkboxes for project configuration. The 'Restrict where this project can be run' checkbox is checked, and the 'Label Expression' field below it contains the text 'master'. A note at the bottom states: 'Label master matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.'

Jenkins / first-job-01 / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Description

added build periodically

Plain text [Preview](#)

- ☐ Enable project-based security
- ☐ Discard old builds ?
- ☐ GitHub project
- ☐ This project is parameterized ?
- ☐ Throttle builds ?
- ☐ Execute concurrent builds if necessary ?
- ☒ Restrict where this project can be run ?

Label Expression ?

master

Label master matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Source Code Management

Navigated to **Source Code Management**.

Selected **Git**.

Entered **Repository URL**:

`https://github.com/srihari03-sh/git-practice.git`

Left **Credentials** as none.

Set **Branch Specifier** to:

`*/main`

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

`https://github.com/srihari03-sh/git-practice.git`

Credentials ?

- none -

+ Add

Advanced ▾

+ Add Repository

Trigger – Build Periodically

Opened **Triggers** section.

Enabled **Build periodically**.

Entered schedule:

`* * * * *`

Saved the job configuration.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☒ Build periodically ?

Schedule ?

* * * * *

⚠ Do you really mean "every minute" when you say "* * * * *"? Perhaps you meant "H * * * * *" to poll once per hour

Would last have run at Wednesday, August 12, 2026, 8:09:00 AM Coordinated Universal Time; would next run at Wednesday, August 12, 2026, 8:10:00 AM Coordinated Universal Time.

☐ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

Jenkins Built Execution

Jenkins automatically triggered builds based on the schedule.

Multiple builds appeared in **Build History**.

Selected a build and opened **Console Output**.

Verified message:

Started by timer

Jenkins fetched the latest code from GitHub.

Build completed with **SUCCESS** status.

SCM Trigger Verification

Returned to **Triggers** configuration.

Enabled **Poll SCM**.

Entered schedule:

* * * * *

Saved configuration.

GitHub Repository Change

Opened GitHub repository **devops-hub**.

Edited **README.md**.

Added content to the file.

Committed changes to **master** branch.

Jenkins SCM-Based Build

Jenkins detected SCM change.

New build appeared with status **Started by an SCM change**.

Opened Console Output.

Verified Git fetch and checkout steps.

Build completed with **SUCCESS**.

 **Jenkins** / first-job-01 / #5

Status

Changes

Console Output

Delete build '#5'

Git Build Data

Edit build information

Previous Build

Console

DownloadCopy

```
Started by timer
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/first-job-01
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/first-job-01/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/srihari03-sh/git-practice.git # timeout=10
Fetching upstream changes from https://github.com/srihari03-sh/git-practice.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/srihari03-sh/git-practice.git +refs/heads/*:refs/remotes/origin
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 97b634a6138559574fc5a3401839801bbe53a1a8 (refs/remotes/origin/main)
```

 **Jenkins** / first-job-01

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

 **first-job-01**

added build periodically

Permalinks

- Last build (#7), 29 sec ago
- Last stable build (#7), 29 sec ago
- Last successful build (#7), 29 sec ago
- Last completed build (#7), 29 sec ago

Builds >

Filter /

Today

-  #7 8:17 AM
-  #6 8:16 AM
-  #5 8:15 AM
-  #4 8:14 AM
-  #3 8:13 AM
-  #2 8:12 AM

 **Jenkins** / first-job-01 / Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

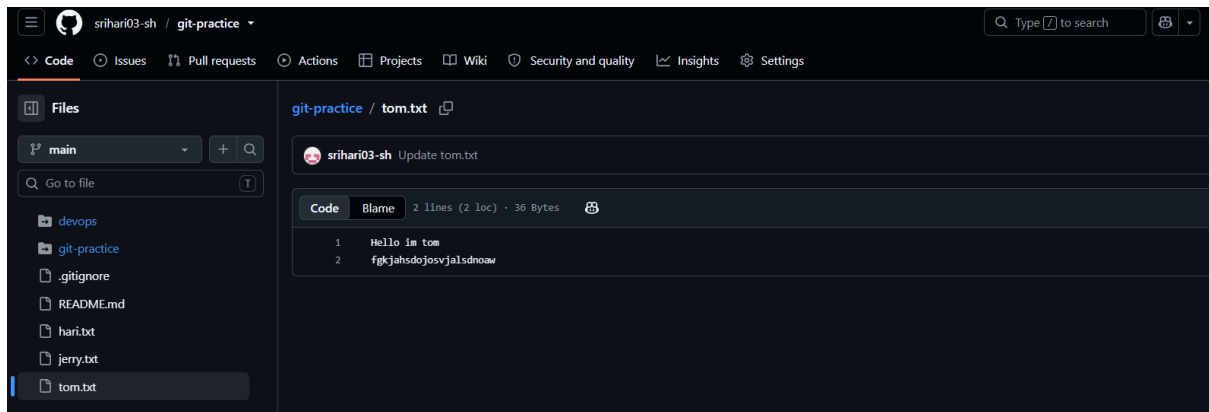
☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

 Do you really mean "every minute" when you say "*****"? Perhaps you meant "H *****" to poll once per hour
Would last have run at Wednesday, August 12, 2026, 8:20:00 AM Coordinated Universal Time; would next run at Wednesday, August 12, 2026, 8:21:00 AM Coordinated Universal Time.

☐ Ignore post-commit hooks ?



 **Jenkins** / first-job-01

Status

</> Changes

Workspace

Build Now

Delete Project

Git Polling Log

Configure

Rename

Builds >

Filter

Pending

#11

Finished waiting

 **first-job-01**

added build periodically

Permalinks

- Last build (#10), 40 min ago
- Last stable build (#10), 40 min ago
- Last successful build (#10), 40 min ago
- Last completed build (#10), 40 min ago

 **Jenkins** / first-job-01 / #11

Status

</> Changes

Console Output

Delete build '#11'

Polling Log

Git Build Data

Edit build information

Previous Build

Console

Download

Copy

```
Started by an SCM change
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/first-job-01
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/first-job-01/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/srihari03-sh/git-practice.git # timeout=10
Fetching upstream changes from https://github.com/srihari03-sh/git-practice.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/srihari03-sh/git-practice.git +refs/heads/*:refs/remotes/origin
```

4. Take backup of Jenkins server by using bash script.

Steps Performed – Jenkins Backup Script

Objective

To take a backup of the Jenkins Master server using a Bash script and securely transfer the backup file to Jenkins Slave/Agent servers using SCP.

Environment

- Jenkins Master: EC2 instance
 - Jenkins Slave-01: EC2 instance
 - Jenkins Slave-02: EC2 instance
 - Jenkins Home Directory: /var/lib/jenkins
 - Backup Directory on Master: /backup/jenkins
 - Backup Format: .tar.gz
-

Step 1: Create the Jenkins Backup Script

Created a Bash script to back up the Jenkins Home directory.

Created the script file:

```
sudo vi /opt/scripts/jenkins_backup.sh
```

Added the following content:

```
#!/bin/bash
```

```
# Jenkins home directory
```

```
JENKINS_HOME="/var/lib/jenkins"
```

```
# Backup directory
```

```
BACKUP_DIR="/backup/jenkins"
```

```
# Date variable
```

```
DATE=$(date +"%Y-%m-%d_%H-%M-%S")
```

```
# Create backup directory
```

```
mkdir -p "$BACKUP_DIR"
```

Take Jenkins backup

```
tar -czvf "$BACKUP_DIR/jenkins_backup_$(date +%Y-%m-%d_%H-%M-%S).tar.gz" \
--exclude="$JENKINS_HOME/workspace" \
--exclude="$JENKINS_HOME/logs" \
"$JENKINS_HOME"
```

echo "Jenkins backup completed successfully!"

echo "Backup file: \$BACKUP_DIR/jenkins_backup_\$(date +%Y-%m-%d_%H-%M-%S).tar.gz"

Explanation

- JENKINS_HOME → Jenkins data directory.
 - BACKUP_DIR → Location where the backup will be stored.
 - DATE → Adds the current date and time to the backup filename.
 - mkdir -p → Creates the backup directory if it does not exist.
 - tar -czvf → Compresses the Jenkins directory into a .tar.gz file.
 - --exclude → Excludes workspace and log directories from the backup.
-

Step 2: Give Execute Permission

After saving the script, provided execute permission:

```
sudo chmod +x /opt/scripts/jenkins_backup.sh
```

Verified the script:

```
ls -l /opt/scripts/jenkins_backup.sh
```

Step 3: Execute the Backup Script

Ran the script:

```
sudo /opt/scripts/jenkins_backup.sh
```

The script completed successfully and generated a backup file similar to:

Jenkins backup completed successfully!

Backup file: /backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz

Step 4: Verify the Backup File

Checked the backup directory:

```
ls -lh /backup/jenkins/
```

Output:

```
jenkins_backup_2026-08-12_09-34-50.tar.gz
```

This confirmed that the Jenkins backup was successfully created.

Transfer Jenkins Backup to Slave-01

Step 5: Configure SSH Authentication

To transfer the backup from the Jenkins Master to Slave-01 using SCP, SSH authentication between the servers was configured.

On the Jenkins Master, generated an SSH key pair:

```
ssh-keygen
```

Pressed **Enter** to accept the default location:

```
/home/ec2-user/.ssh/id_rsa
```

The following files were created:

```
/home/ec2-user/.ssh/id_rsa
```

```
/home/ec2-user/.ssh/id_rsa.pub
```

- id_rsa → Private key
 - id_rsa.pub → Public key
-

Step 6: Add Master Public Key to Slave-01

Displayed the public key on the Master:

```
cat ~/.ssh/id_rsa.pub
```

Copied the complete public key.

Logged into Slave-01 using its EC2 access method and created the SSH directory:

```
mkdir -p ~/.ssh
```

```
chmod 700 ~/.ssh
```

Opened the authorized keys file:

```
vi ~/.ssh/authorized_keys
```

Pasted the Master public key into the file.

Then set the correct permission:

```
chmod 600 ~/.ssh/authorized_keys
```

Step 7: Test Passwordless SSH

From the Jenkins Master, tested the connection:

```
ssh ec2-user@44.222.162.151
```

The connection was successful without requiring the original EC2 private key each time.

This confirmed that passwordless SSH authentication between Master and Slave-01 was working.

Transfer the Backup Using SCP

Step 8: Create Backup Directory on Slave-01

Initially, SCP returned the following error:

```
scp: dest open "/backup/jenkins/": No such file or directory
```

This occurred because the destination directory did not exist on Slave-01.

Logged into Slave-01:

```
ssh ec2-user@44.222.162.151
```

Created the directory:

```
sudo mkdir -p /backup/jenkins
```

Changed ownership so that ec2-user could write to the directory:

```
sudo chown -R ec2-user:ec2-user /backup/jenkins
```

Verified:

```
ls -ld /backup/jenkins
```

Step 9: Copy Backup from Master to Slave-01

Returned to the Jenkins Master and executed:

```
scp /backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz \  
ec2-user@44.222.162.151:/backup/jenkins/
```

The backup file was transferred from the Jenkins Master to Slave-01.

SCP Explanation

Source:

```
/backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz
```

Destination:

ec2-user@44.222.162.151:/backup/jenkins/

Step 10: Verify Backup on Slave-01

Logged into Slave-01:

```
ssh ec2-user@44.222.162.151
```

Checked the backup directory:

```
ls -lh /backup/jenkins/
```

The backup file was present:

```
jenkins_backup_2026-08-12_09-34-50.tar.gz
```

This confirmed that the Jenkins backup was successfully transferred from the Master to Slave-01.

Important SCP Error and Resolution

Initially, the following command was used incorrectly:

```
scp -i jenkins_backup_2026-08-12_09-34-50.tar.gz ec2-user@44.222.162.151:~
```

This was incorrect because **-i** is used to specify an **SSH private key**, not the backup file.

The correct approach was to configure SSH authentication and then use:

```
scp /backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz \
ec2-user@44.222.162.151:/backup/jenkins/
```

Final Result

The Jenkins backup process was successfully configured:

1. Jenkins Home directory was identified.
2. Bash backup script was created.
3. Execute permission was provided.
4. Jenkins backup was generated as a .tar.gz file.
5. Backup file was verified on the Master.
6. SSH key-based authentication was configured between Master and Slave-01.
7. Backup directory was created on Slave-01.
8. Backup was transferred using SCP.
9. Backup file was verified on Slave-01.

Next step: Repeat the same SCP process for **Slave-02** so that the Jenkins backup is stored on both slave servers.

```
Last login: Wed Aug 12 06:44:27 2026 from 18.206.107.28
[ec2-user@ip-172-31-4-173 ~]$ sudo su
[root@ip-172-31-4-173 ec2-user]# cd /opt/
[root@ip-172-31-4-173 opt]# ls
aws
[root@ip-172-31-4-173 opt]# pwd
/opt
[root@ip-172-31-4-173 opt]# mkdir -p /opt/scripts
[root@ip-172-31-4-173 opt]# cd /opt/scripts
[root@ip-172-31-4-173 scripts]# vi jenkins backup.sh

#!/bin/bash

# Jenkins home directory
JENKINS_HOME="/var/lib/jenkins"

# Backup directory
BACKUP_DIR="/backup/jenkins"

# Date variable
DATE=$(date +%Y-%m-%d_%H-%M-%S)

# Create backup directory
mkdir -p "$BACKUP_DIR"

# Take Jenkins backup
tar -czvf "$BACKUP_DIR/jenkins_backup_${DATE}.tar.gz" \
  --exclude="$JENKINS_HOME/workspace" \
  --exclude="$JENKINS_HOME/logs" \
  "$JENKINS_HOME"

echo "Jenkins backup completed successfully!"
echo "Backup file: $BACKUP_DIR/jenkins_backup_${DATE}.tar.gz"

/var/lib/jenkins/nodes/slave-01/
/var/lib/jenkins/nodes/slave-01/config.xml
/var/lib/jenkins/nodes/slave-02/
/var/lib/jenkins/nodes/slave-02/config.xml
/var/lib/jenkins/queue.xml.bak
/var/lib/jenkins/queue.xml
Jenkins backup completed successfully!
Backup file: /backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz
[root@ip-172-31-4-173 scripts]# cd /opt
[root@ip-172-31-4-173 opt]# ls
aws scripts
[root@ip-172-31-4-173 opt]# cd /scripts
bash: cd: /scripts: No such file or directory
[root@ip-172-31-4-173 opt]# cd /scripts/
bash: cd: /scripts/: No such file or directory
[root@ip-172-31-4-173 opt]# cd /backup/jenkins
[root@ip-172-31-4-173 jenkins]# ls
jenkins_backup_2026-08-12_09-34-50.tar.gz
[root@ip-172-31-4-173 jenkins]#

[ec2-user@ip-172-31-4-173 jenkins]$ scp /backup/jenkins/jenkins_backup_2026-08-12_09-34-50.tar.gz ec2-user@44.222.162.151:/backup/jenkins/
jenkins backup 2026-08-12_09-34-50.tar.gz                                100% 144MB 177.7MB/s   00:00
[ec2-user@ip-172-31-4-173 jenkins]$

i-038802f91ba3b02fa (Jenkin server)
PublicIPs: 98.86.150.73  PrivateIPs: 172.31.4.173
```

```
Last login: Wed Aug 12 10:22:48 2026 from 98.86.150.73
[ec2-user@ip-172-31-7-219 ~]$ sudo mkdir -p /backup/jenkins
[ec2-user@ip-172-31-7-219 ~]$ sudo chown -R ec2-user:ec2-user /backup/jenkins
[ec2-user@ip-172-31-7-219 ~]$ ls -ld /backup/jenkins
drwxr-xr-x. 2 ec2-user ec2-user 6 Aug 12 10:24 /backup/jenkins
[ec2-user@ip-172-31-7-219 ~]$ ls -lh /backup/jenkins/
total 144M
-rw-r--r--. 1 ec2-user ec2-user 144M Aug 12 10:26 jenkins_backup_2026-08-12_09-34-50.tar.gz
[ec2-user@ip-172-31-7-219 ~]$

i-0fb51916fc7a9d825 (slave-01)
PublicIPs: 44.222.162.151  PrivateIPs: 172.31.7.219
```

5. Take backup of Jenkins using thin backup plugin.

Steps Performed – Jenkins ThinBackup Plugin Backup

Install ThinBackup Plugin

Opened Jenkins UI.

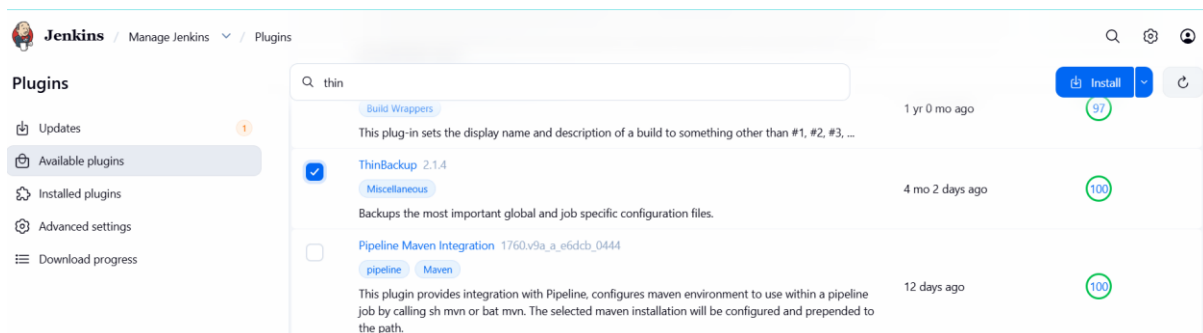
Navigated to:

Manage Jenkins → Plugins → Available plugins

Searched for **ThinBackup**.

Selected the **ThinBackup** plugin.

Clicked **Install**.



Plugin Installation Verification

Opened **Download progress** page.

Verified:

- Internet connectivity check → Success
- Update center connectivity → Success
- ThinBackup plugin → Installed successfully

Clicked **Go back to the top page**.

Access ThinBackup

Navigated to:

Manage Jenkins → ThinBackup

Confirmed ThinBackup page loaded successfully.

Observed available options:

- **Backup now**
- **Restore**

Prepare Backup Directory on Server

Logged in to Jenkins server as root.

Created backup directory:

```
mkdir -p /opt/jenkins-backup
```

Changed ownership to Jenkins user:

```
chown -R jenkins:jenkins /opt/jenkins-backup
```

Set permissions:

```
chmod 755 /opt/jenkins-backup
```

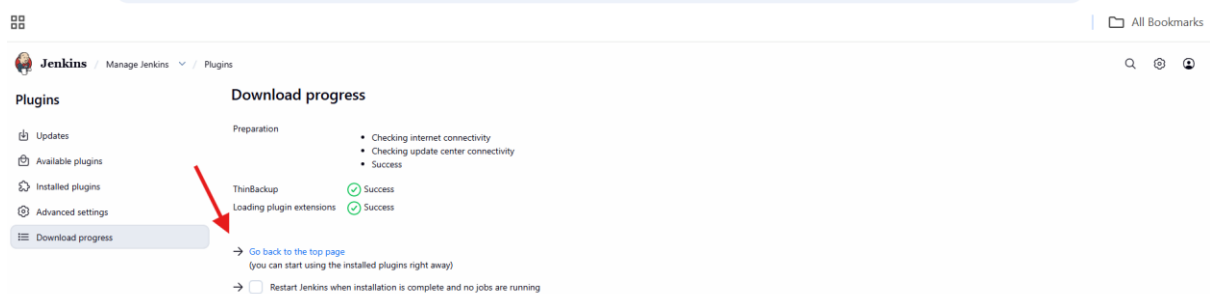
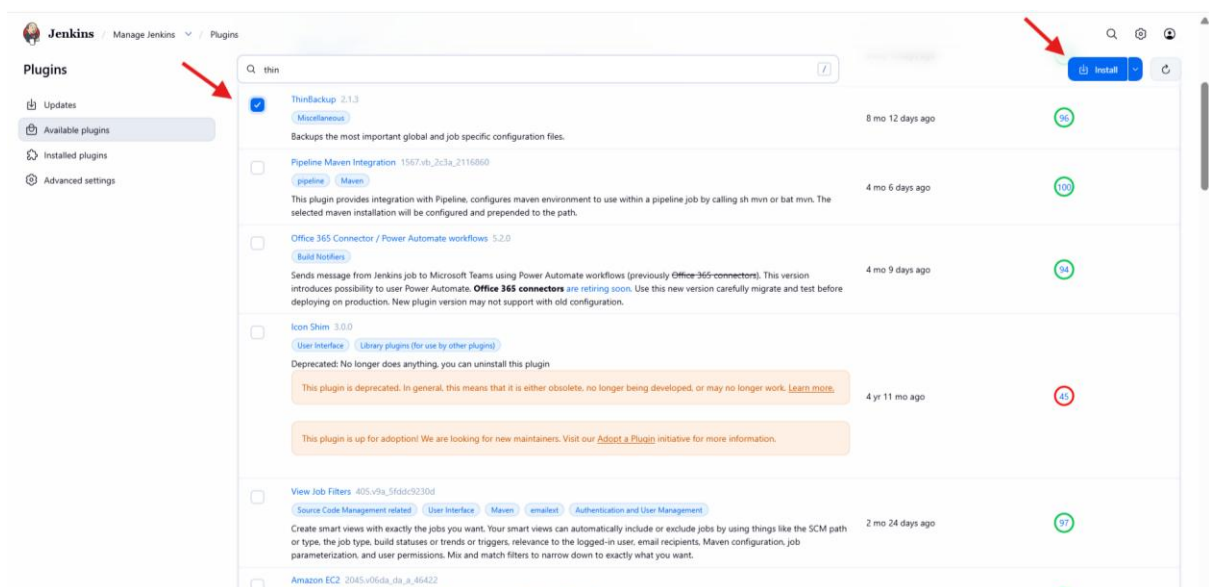
Verify Backup File

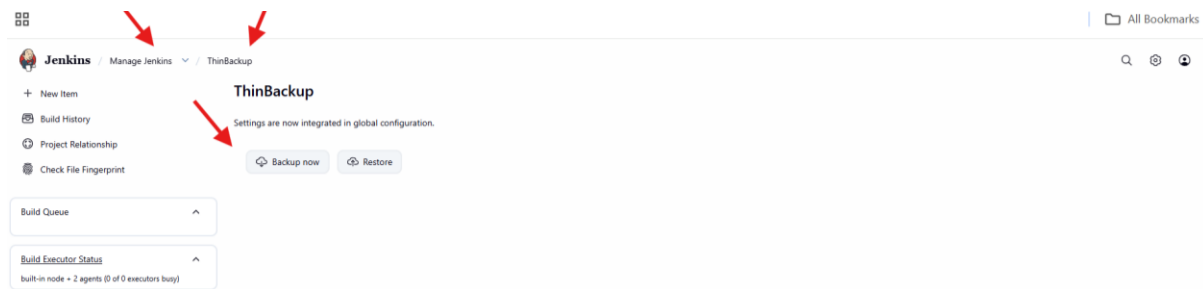
Listed contents of backup directory:

```
ls -ltr /opt/jenkins-backup
```

Verified backup archive exists:

```
jenkins_backup_2025-12-25_13-57.tar.gz
```





```
[root@ip-172-31-40-64 /]# mkdir -p /opt/jenkins-backup
[root@ip-172-31-40-64 /]# chown -R jenkins:jenkins /opt/jenkins-backup
[root@ip-172-31-40-64 /]# chmod 755 /opt/jenkins-backup
[root@ip-172-31-40-64 /]# ls -ltr /opt/jenkins-backup
total 142536
-rw-r--r--. 1 jenkins jenkins 145956760 Dec 25 13:58 jenkins_backup_2025-12-25_13-57.tar.gz
[root@ip-172-31-40-64 /]#
```

6. Setup a new Jenkins server and dump the backup taken in task4.

On Jenkins Master (Old Server)

Logged in to **jenkins-master** EC2 instance.

Navigated to /opt:

```
cd /opt
```

Listed directories and verified jenkins-backup exists.

Moved into backup directory:

```
cd jenkins-backup
```

Verified Jenkins backup file is present:

```
jenkins_backup_2025-12-25_13-57.tar.gz
```

On Backup EC2 Instance (New Server)

Logged in to **backup EC2 instance**.

Added Jenkins repository:

```
wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat/jenkins.repo
```

Imported Jenkins GPG key:

```
rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

Installed Java:

```
yum install java-17-amazon-corretto -y
```

Installed Jenkins:

```
yum install jenkins -y
```

Started Jenkins service:

```
systemctl start jenkins
```

Verified Jenkins service status:

```
systemctl status jenkins
```

Access Jenkins on Backup Server

Opened browser and accessed Jenkins:

<http://56.228.32.222:8080>

Confirmed Jenkins UI loaded successfully.

Transfer Backup File to Backup Server

Returned to **jenkins-master**.

Changed permission on backup file:

Created a file test.pem and copied pem key

```
chmod 600 test.pem
```

Copied backup file to backup server:

```
scp -i test.pem jenkins_backup_2025-12-25_13-57.tar.gz ec2-user@56.228.32.222:/tmp
```

Accepted SSH host key when prompted.

Verified file transfer completed successfully.

Restore Backup on Backup Server

Logged in to **backup EC2 instance**.

Navigated to /tmp:

```
cd /tmp
```

Verified backup file exists:

jenkins_backup_2025-12-25_13-57.tar.gz

Extracted backup into Jenkins home:

```
tar -xvf jenkins_backup_2025-12-25_13-57.tar.gz -C /var/lib/jenkins
```

Verified Jenkins files restored under /var/lib/jenkins.

Restart Jenkins

Restarted Jenkins using browser:

<http://56.228.32.222:8080/restart>

Jenkins restarted successfully.

```
[ec2-user@ip-172-31-5-51 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Wed Aug 12 12:15:41 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-5-51 ~]$ sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:00:24 ago on Wed Aug 12 12:15:41 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository
Installing: java-21-amazon-corretto	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux
Installing dependencies: alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.3	amazonlinux
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux

```
[ec2-user@ip-172-31-5-51 ~]$ sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key
[ec2-user@ip-172-31-5-51 ~]$ sudo dnf install jenkins -y
Jenkins-stable
Jenkins-stable
Importing GPG key 0x14ABFC68:
  Userid   : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From      : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable
Dependencies resolved.
```

Package	Architecture	Version	Repository
Installing: jenkins	noarch	2.568.2-1	jenkins

```
[ec2-user@ip-172-31-5-51 ~]$ sudo systemctl enable --now jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[ec2-user@ip-172-31-5-51 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-08-12 12:17:14 UTC; 1s ago
     Main PID: 29645 (java)
       Tasks: 45 (limit: 4543)
      Memory: 606.6M
         CPU: 12.985s
    CGroup: /system.slice/jenkins.service
            └─29645 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
```

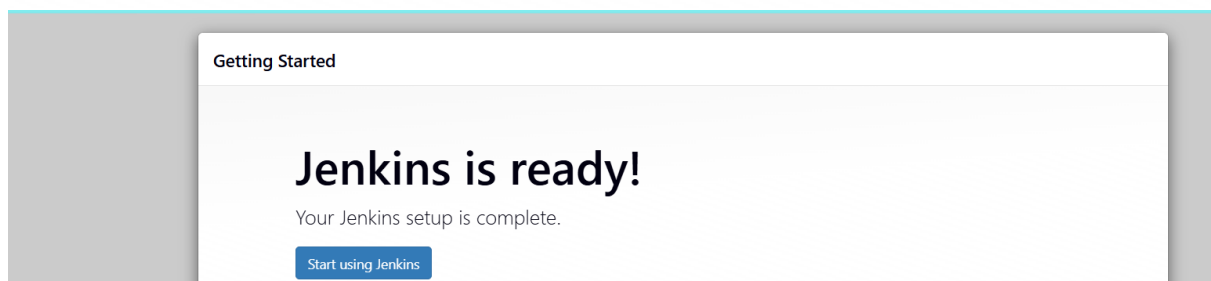
Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`/var/lib/jenkins/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password



```
[root@ip-172-31-40-64 jenkins]# cd
[root@ip-172-31-40-64 ~]# cd /opt/
[root@ip-172-31-40-64 opt]# ls
aws jenkins-backup sssutils
[root@ip-172-31-40-64 opt]# cd jenkins-backup/
[root@ip-172-31-40-64 jenkins-backup]# ls
jenkins-backup-2025-12-25-15-57.tar.gz
[root@ip-172-31-40-64 jenkins-backup]#
```

i-00eec06bc53ee2df4 (jenkins-master)
PublicPc: 13.53.212.231 PrivatePc: 172.31.40.64

```
[root@ip-172-31-45-140 ~]# wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat/jenkins.repo
--2025-12-25 14:57:20-- https://pkg.jenkins.io/redhat/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 159.232.174.133, 2a04:4e42:6b::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|159.232.174.133|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://pkg.jenkins.io/rpm/jenkins.repo [following]
--2025-12-25 14:57:20-- https://pkg.jenkins.io/rpm/jenkins.repo
Resuming existing connection to pkg.jenkins.io:443.
HTTP request sent, awaiting response... 200 OK
Length: 246 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo 100%[=====] 246 --.-KB/s in 0s

2025-12-25 14:57:21 (13.5 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [246/246]

[root@ip-172-31-45-140 ~]# rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
[root@ip-172-31-45-140 ~]# yum upgrade
amazon linux 2023 kernel livepatch repository 246 kb/s | 29 kb 00:00
jenkins 1.6 kb/s | 833 b 00:00
jenkins 3.7 kb/s | 1.6 kb 00:00
Importing GPG key 0x144BFC68:
Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
Fingerprint: 5828 c8a0 8358 0150 c2ac 8ac9 7188 f480 142a 9c49
From : https://pkg.jenkins.io/rpm/repodata/repomd.xml.key
Is this ok [y/N]: y
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-45-140 ~]# yum install java-17-amazon-corretto -y
Last metadata expiration check: 0:00:07 ago on Thu Dec 25 14:59:06 2025.
Dependencies resolved.

Package Architecture Version Repository Size
Installing:
java-17-amazon-corretto x86_64 1:17.0.17+10-1.amzn2023.1 amazonlinux 219 k
Installing dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.2 amazonlinux 504 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
harfbuzz x86_64 2.9.1-16.amzn2023.0.2 amazonlinux 1.3 M
```

i-0e79044f52a854fb3 (backup)
PublicPc: 56.228.32.222 PrivatePc: 172.31.45.140

```
Complete!
[root@ip-172-31-45-140 ~]# systemctl start jenkins
[root@ip-172-31-45-140 ~]# systemctl status jenkins
* jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Active: active (running) since Thu 2025-12-25 15:01:12 UTC; 17s ago
     Main PID: 26085 (java)
       Tasks: 45 (limit: 1067)
      Memory: 351.0M
         CPU: 18.346s
    CGroup: /system.slice/jenkins.service
            └─26085 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Dec 25 15:01:07 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: [L#]>
Dec 25 15:01:07 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: [L#]> *****
Dec 25 15:01:07 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: [L#]> *****
Dec 25 15:01:07 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: [L#]> *****
Dec 25 15:01:12 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: 2025-12-25 15:01:12.787+0000 [id=32] INFO jenkins.InitReactorRunner$1onAttained: Completed initialization
Dec 25 15:01:12 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: 2025-12-25 15:01:12.806+0000 [id=23] INFO Hudson.lifecycle.Lifecycle$onReady: Jenkins is fully up and running
Dec 25 15:01:12 ip-172-31-45-140.eu-north-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Dec 25 15:01:13 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: 2025-12-25 15:01:13.523+0000 [id=49] INFO hudson.DownloadService$Downloadable$load: Obtained the updated data file for h
Dec 25 15:01:13 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: 2025-12-25 15:01:13.523+0000 [id=49] INFO hudson.util.Retrier$start: Performed the action check updates server success
Dec 25 15:01:17 ip-172-31-45-140.eu-north-1.compute.internal jenkins[26085]: 2025-12-25 15:01:17.860+0000 [id=69] WARNING hudson.slaves.DiskSpaceMonitor$Descriptor$MarkNodeOfflineOrOnline: Making Built-In
lines 1-20/20 (END)
```

i-0e79044f52a854fb3 (backup)
PublicPc: 56.228.32.222 PrivatePc: 172.31.45.140

← → ↺

⚠ Not secure 56.228.32.222:8080

☆ 📁 M ⋮

📁 All Bookmarks

 **Jenkins**

+ New Item

📜 Build History

🔍 ⚙️ 👤

Add description

Build Queue

▼

No builds in the queue.

Build Executor Status

0/2 ▼

 Built-In Node (offline)

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent 🖥

Configure a cloud ☁

Learn more about distributed builds ?

```
[root@ip-172-31-45-140 ~]# cd /tmp/
[root@ip-172-31-45-140 tmp]# ls
superfdata_jenkins
jenkins_backup_2025-12-25_13-57.tar.gz
jesty-0.0.0-0-8080-war-*.any-4676708329651187170
systemd-private-602b20c0d4f14471874cd229b885e240-chrompd.service-cAp001
systemd-private-602b20c0d4f14471874cd229b885e240-dbus-broker.service-Pd9tG0
[root@ip-172-31-45-140 tmp]# cp jenkins_backup_2025-12-25_13-57.tar.gz /war/lib/
cp: overwrite '/war/lib/jenkins_backup_2025-12-25_13-57.tar.gz'? yes
[root@ip-172-31-45-140 tmp]# cd /war/lib/
[root@ip-172-31-45-140 lib]# ls
alternatives chrompy dnf gssproxy initramfs jenkins_backup_2025-12-25_13-57.tar.gz logrotate nfs portables rpm selinux systemd war yum
bazoon cloud qpsos libdnf-report jenkins kparton misc os-prober private rpm-state sss update-motd xfsdump
[root@ip-172-31-45-140 lib]# tar xvf jenkins_backup_2025-12-25_13-57.tar.gz
```

i-0e79044f52a854fb3 (backup)
PublicIPs: 56.228.32.222 PrivateIPs: 172.31.45.140

